系统设计说明书

大学生课程课件笔记整理与知识梳理 Agent

版本 V1.1 | 2026 年 11 月

项目	内容
文档名称	系统设计说明书（System Design Specification）
对应系统	大学生课程课件笔记整理与知识梳理 Agent
线上地址	http://47.114.55.115:8000
文档状态	与当前代码实现一致（模块、接口、数据表、阈值均取自实际代码）
编写依据	《需求分析说明书》、Figma 原型设计稿、实际代码与自动化测试结果
编写	编程组（组长：徐张睿）
适用读者	开发、测试、答辩评审、后续维护者

修订记录

版本	日期	修订内容	修订人
V1.0	2026-11-08	首次成稿：覆盖架构、模块、接口、数据、流程、非功能与部署	编程组
V1.1	2026-11-08	补充目录、页眉页码、状态机与调用示例；订正接口与数据表计数口径	编程组

目 录

未找到目录项。

1 引言

1.1 编写目的

本文描述系统的总体结构、模块划分、接口与数据结构、关键流程与非功能设计，供开发人员维护、测试人员验收、答辩与评审时查阅。文中所有量化指标均来自实际代码、自动化测试与线上真实调用，未采用估算值。

1.2 设计范围

本期实现的是「单课程闭环」：上传课件 → 解析分块 → 生成带出处的结构化笔记 → 导出 → 基于材料问答。领域知识库（kb.db）仅建立表结构，未灌入数据；智能自测与错题本处于设计阶段，未实现（详见第 11 章）。

1.3 术语与缩写

术语	含义
分块（chunk）	解析后的一段连续原文，带页码/幻灯片号与章节归属，是检索与引用的最小单位
出处（evidence）	一条知识点对应的原文片段，含 chunk_key、文件名、页码与连续引文
证据回查	把模型给出的引文放回原文块中查找，查不到即丢弃该知识点
置信三态	检索结果分为 confident / low_confidence / empty，问答据此决定能否作答
质量自评	按核心 85%、结构 10%、关联 5% 加权得到 0~100 分的自查分数
Envelope	统一的响应包络 {code, message, data}

1.4 参考资料

- 《需求分析说明书》（含 GB/T 9385-2008 结构）
- 原型设计稿（Figma 交互原型 agent2.0）：https://www.figma.com/proto/iDdth59yZaOjSaWgHvajkQ/agent2.0
- 本项目 README、交接说明与技术决策记录
- 阿里云 ECS 部署环境（Ubuntu 22.04）

2 系统概述

2.1 建设目标

学生上传课程课件后，系统自动产出一份「可追溯到原文」的结构化笔记：包含章节大纲、知识点定义、要点、重点清单与文本思维导图，并能就材料内容问答。核心约束是——所有结论必须能指回课件的具体页码与原文，查不到依据的内容一律丢弃。

2.2 角色与用例

角色	主要用例
学生用户	注册/登录、上传课件、发起整理、查看进度、查看笔记与出处、导出（Word/Markdown/LaTeX）、补充材料并合并、课程问答、查看历史任务
管理员	查看统计、维护公共课程资料、查看效果对比记录

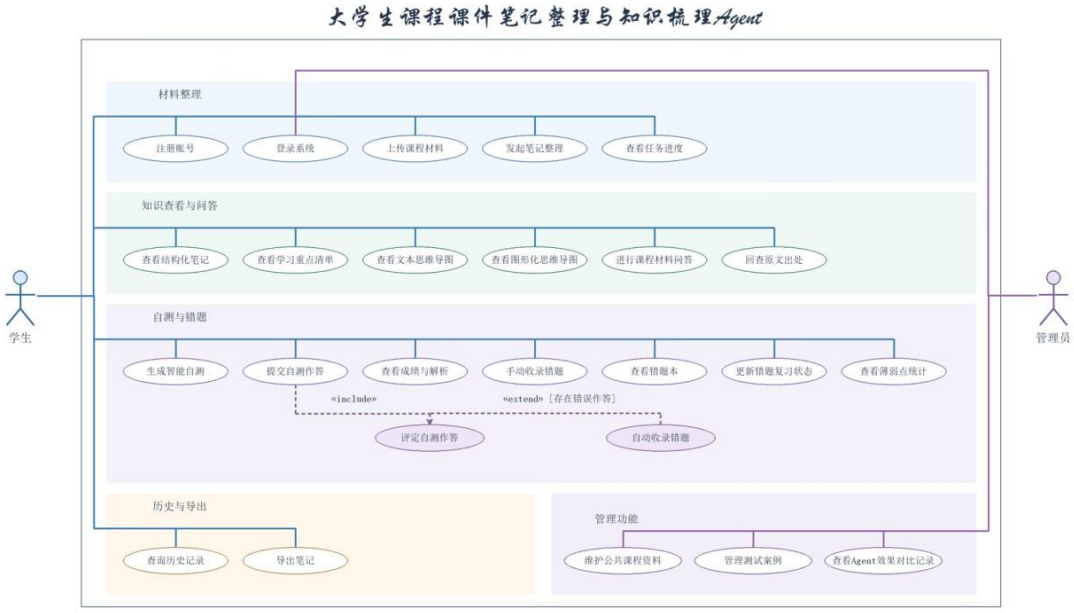


图 1 系统用例图（学生/管理员两类角色）

2.3 系统边界

在范围内	不在本期范围
PDF/PPTX/DOCX/TXT/MD/HTML/CSV/EPUB 的解析与笔记生成	领域知识库的数据填充（表结构已建，未灌数据）
证据绑定、引文回查、无依据内容丢弃	智能自测、错题本（仅设计）
课程问答（材料不足时如实说明）	手写公式识别 OCR（接口预留，未启用）

在范围内	不在本期范围
Word / Markdown / LaTeX 三种导出	多用户协作编辑、移动端 App
多材料合并整理、补充材料	向量检索（配置项保留，当前关闭）

2.4 运行环境

项	配置
服务器	阿里云 ECS 2 核 4G，Ubuntu 22.04，公网带宽 5 Mbps
运行时	Python 3.12 + uvicorn（单 worker）+ systemd 守护
存储	SQLite（业务库 + 知识库，物理分离，WAL 模式）
前端	Vue 3 + Vite + Element Plus，构建产物由后端同源托管
模型	DeepSeek（OpenAI 兼容协议），经 HTTP API 调用

3 总体设计

3.1 分层架构

系统分为五层：前端交互层、后端服务层（API 网关 + 会话/任务编排）、Agent 智能层、工具调用层、数据层。前端不直接接触模型与数据库，所有能力经后端 API 暴露。



图 2 Agent 集成架构图（五层结构）

3.2 模块职责

层次	模块	职责	代码位置
前端	页面与路由	登录、上传、历史、进度、结果、问答六个页面	frontend/src/views

层次	模块	职责	代码位置
前端	请求封装	统一 baseURL、Cookie/Bearer 双鉴权、错误提示、blob 处理	frontend/src/utils/http.js
后端	认证与鉴权	注册、登录、会话签名、越权隔离	app/core/security.py、app/api/routes_auth.py
后端	上传与解析	格式嗅探、抽取、分块、入库、建索引	app/services/ingest.py、parse.py
后端	任务编排	创建任务、后台执行、进度与状态机	app/api/routes_sessions.py
Agent	笔记管线	逐块抽取 → 证据校验 → 去重 → 补全 → 渲染 → 自评	app/agents/pipeline.py
Agent	提示词	抽取/渲染/问答/自评四套模板	app/agents/prompts/
工具	检索	FTS5 粗召回 + 二元组覆盖率重排 + 三态判定	app/services/query.py
工具	导出	Markdown / LaTeX / Word 三种渲染	app/api/routes_sessions.py、app/services/export_docx.py
数据	仓储	业务表读写、结果组装	app/db/repo_business.py

3.3 对象协作关系

学生、前端、后端、Agent、数据层之间的消息传递关系如下图：一次整理任务从提交到返回结构化结果，跨四个进程内组件与两次数据库读写。

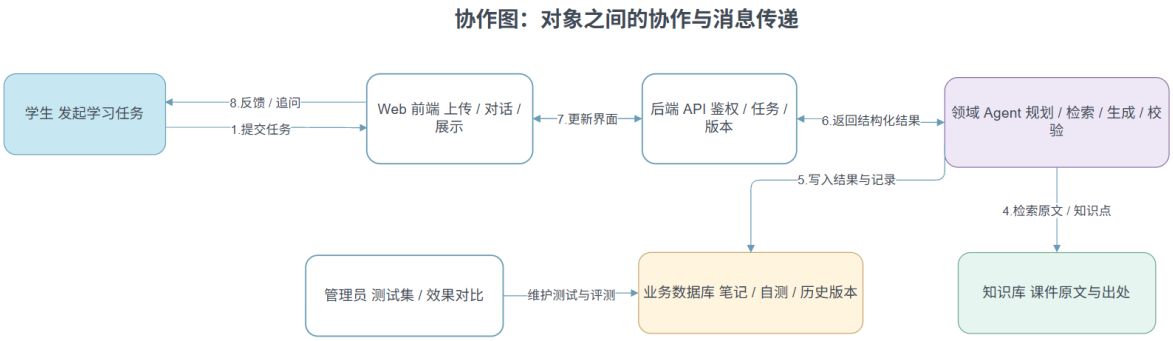


图 3 对象协作图：对象之间的协作与消息传递

3.4 技术选型与理由

选型	理由	代价
FastAPI	自动生成 OpenAPI 文档、Pydantic 校验、异步友好	生态较 Flask 小
SQLite + WAL	单机零运维、事务可靠、备份即拷文件	并发写弱，只适合单机
FTS5 + 二元组	中文无需分词器、可解释、零额外服务	语义召回弱于向量检索
DeepSeek API	中文教材文本表现稳定、价格低	依赖外部网络与密钥
Vue 3 + Element Plus	开发快、组件全、构建产物可直接同源托管	包体较大
systemd	开机自启、崩溃自动重启、日志集中	无进程级监控指标

4 详细设计

4.1 前端交互层

4.1.1 路由与页面

路由	页面	职责
/login、/register	LoginView / RegisterView	登录注册，成功后写入登录态
/upload	UploadView	课程名（可选可输入）、文件校验、上传并创建任务；支持 ?append= 追加模式
/sessions	SessionsView	历史任务列表、状态标签、查看进度/结果、补充材料
/sessions/:id/progress	ProgressView	1.5 秒轮询任务状态，展示真实进度与阶段文案
/sessions/:id/result	ResultView	章节分组展示知识点、出处角标、质量自评、三种导出
/chat	ChatView	课程问答，材料不足时如实提示，不编造

4.1.2 前端约定

- 接口基址固定 /api，生产环境与后端同源，无需 CORS。
- 请求封装统一剥掉 Envelope 外层：调用方拿到的就是 data。
- 下载类请求用 responseType: 'blob'，封装层对 blob 不做 JSON 解析。
- 上传前按 /api/meta 返回的 allowed_extensions 与 max_upload_mb 做前置校验，避免把明显不合规的文件传到后端。
- 失败一律如实提示：早期版本曾在问答接口失败时返回写死的假答案，已删除。

4.2 后端服务层（API）

全部接口共 22 个，统一前缀 /api，统一响应包络 {code, message, data}。鉴权支持 Cookie 与 Bearer 两种方式；所有资源接口都校验归属，越权返回 404 而非 403，避免泄露资源是否存在。

方法	路径	说明
POST	/api/auth/register	注册并签发会话
POST	/api/auth/login	登录
POST	/api/auth/logout	退出
GET	/api/auth/me	当前用户信息
GET	/api/meta	系统能力声明：格式白名单、大小上限、功能开关、登录态
GET	/api/health	健康检查
POST	/api/uploads	上传并解析，返回文档与解析统计
GET	/api/uploads	我的文档列表
GET	/api/uploads/courses	我已上传过的课程名（供前端下拉）
GET	/api/uploads/{doc_id}	文档详情与解析结果
POST	/api/sessions	创建整理任务（支持多份材料）

方法	路径	说明
GET	/api/sessions	我的任务列表
GET	/api/sessions/{id}	任务状态与进度
GET	/api/sessions/{id}/result	结构化结果（含出处与自评）
GET	/api/sessions/{id}/export	导出 md / tex / docx
GET	/api/sessions/{id}/chunks/search	检索调试：查看证据如何被召回
POST	/api/chat/ask	基于材料问答
GET	/api/chat/history	问答历史
GET	/api/admin/stats	管理端统计
GET	/api/admin/materials	公共课程资料列表
POST	/api/admin/materials	新增/更新公共课程资料
GET	/api/admin/compare-records	效果对比记录

4.3 Agent 智能层

4.3.1 笔记生成管线（六段）

管线按「块」处理而非整篇一次性投喂，好处是单块失败不影响整单，坏块只记告警并跳过。

阶段	进度	动作	失败处理
① 读取分块	5%	按任务的材料集合取全部分块	无分块则任务失败并说明原因
② 逐块抽取	10%~45%	每块调用抽取提示词，产出候选知识点	单块失败记告警继续
③ 证据绑定	—	每条候选必须带 chunk_key + 连续引文，回查原文	查不到即丢弃，计入 dropped_items
④ 去重归并	50%	按知识点名称归一化合并，取更长定义、并集要点与证据	—
⑤ 检索补全	58%	用知识点名回检材料，补充关联证据	检索为空则跳过
⑥ 渲染与自评	68%~94%	生成大纲、要点、思维导图；按 85/10/5 自评	渲染失败走确定性兜底

4.3.2 提示词设计

模板	作用	关键约束
extract.txt	从单块原文抽取知识点	只允许使用原文内容；必须给出连续引文；输出严格 JSON
render_note.txt	把知识点组织成大纲与重点	不得引入材料外知识；保持章节层级
qa.txt	基于证据片段作答	只能引用给定片段；无法回答时明确说未涉及
self_eval.txt	按三维度自评	输出 core / structure / extension 三项分数与理由

4.3.3 可靠性设计

- JSON 三重兜底：直接解析 → 提取代码块 → 截取首尾花括号，全部失败才抛错。

- 多密钥轮换：LLM_API_KEYS 支持逗号分隔多把，单把失效自动换下一把，并做指数退避。
- 占位实现：未配置密钥或设置 AGENT_STUB=1 时走 stub 管线，保证整条链路可演示、可测试。
- 质量自评兜底：模型自评失败时用确定性规则（有出处比例、结构完整度）算分，不虚构分数。

4.4 工具调用层

4.4.1 多格式解析

解析器按「文件头嗅探」识别真实格式，不信任扩展名；同名扩展名映射到同一解析器。当前注册 9 类解析器、对外声明 16 种扩展名。

解析器	扩展名	实现要点
pdf	.pdf	PyMuPDF 取文本层；保留页码；扫描件按每页字数判定
pptx	.pptx	python-pptx 逐页取文本；保留幻灯片序号
docx	.docx	python-docx 取段落，按标题样式识别章节
txt	.txt .text	按空行与句子边界切段
md	.md .markdown	识别 # 层级作为章节
html	.html .htm	去标签、保留块级结构
csv	.csv	逐行转文本，保留表头作为上下文
epub	.epub	按章节解压取正文
image	.png .jpg .jpeg .bmp .webp	OCR 接入层（需配置密钥，未配置时明确报错）

4.4.2 检索算法

采用「粗召回 + 精排」两段式，全部在 SQLite 内完成，无可解释性损失：

步骤	做法	目的
① 粗召回	FTS5 建二元组索引（ft_chunks），按查询词做 OR 召回	快速缩小候选集
② 覆盖率重排	计算查询词在分块中的覆盖比例，按覆盖率降序	避免「命中一个高频词」就被判相关
③ 三态判定	覆盖率 ≥ 0.40 记 confident；0.25~0.40 记 low_confidence；< 0.25 丢弃	让问答能区分「没涉及」与「依据不足」

阈值 MIN_COVERAGE=0.25、LOW_CONFIDENCE=0.40 在 app/services/query.py 中定义，改动后需重跑问答评测集确认效果。

4.4.3 导出

格式	实现	适用场景	注意点
Markdown	模板拼接	贴博客、GitHub、Notion	纯文本，公式用 $$ 包裹
LaTeX	模板拼接	排版公式、论文	中文标题需转义
Word(.docx)	python-docx 结构化渲染	交作业、报告	必须清洗 XML 控制字符（PPT 文本含 \x0b 会崩）

三种导出的文件名统一走 RFC 5987 的 filename*，否则中文课程名会让响应头编码抛异常。

4.5 数据层

业务库共 17 张表，构成为：10 张业务表（users、documents、chunks、sessions、notes、note_items、note_versions、chats、example_records、formula_records）+ 1 个 FTS5 虚拟表 ft_chunks + 5 张 FTS5 影子表 + sqlite_sequence。知识库 kb.db 独立成库，共 5 张表，由管理员维护、Agent 只读。

库	表	用途
business.db	users	账号、口令哈希、角色
business.db	documents	上传文档、解析状态、页数与分块数
business.db	chunks	分块原文、页码/幻灯片号、章节归属
business.db	ft_chunks（+5 张影子表）	FTS5 全文索引，按二元组建索引
business.db	sessions	整理任务：状态、进度、阶段文案、材料集合
business.db	notes / note_items / note_versions	笔记正文、知识点条目、版本快照
business.db	chats	问答历史
business.db	example_records / formula_records	例题与公式记录（预留）
kb.db	knowledge_points 等 5 张	领域知识库（结构已建，未灌数据）

业务概念类图

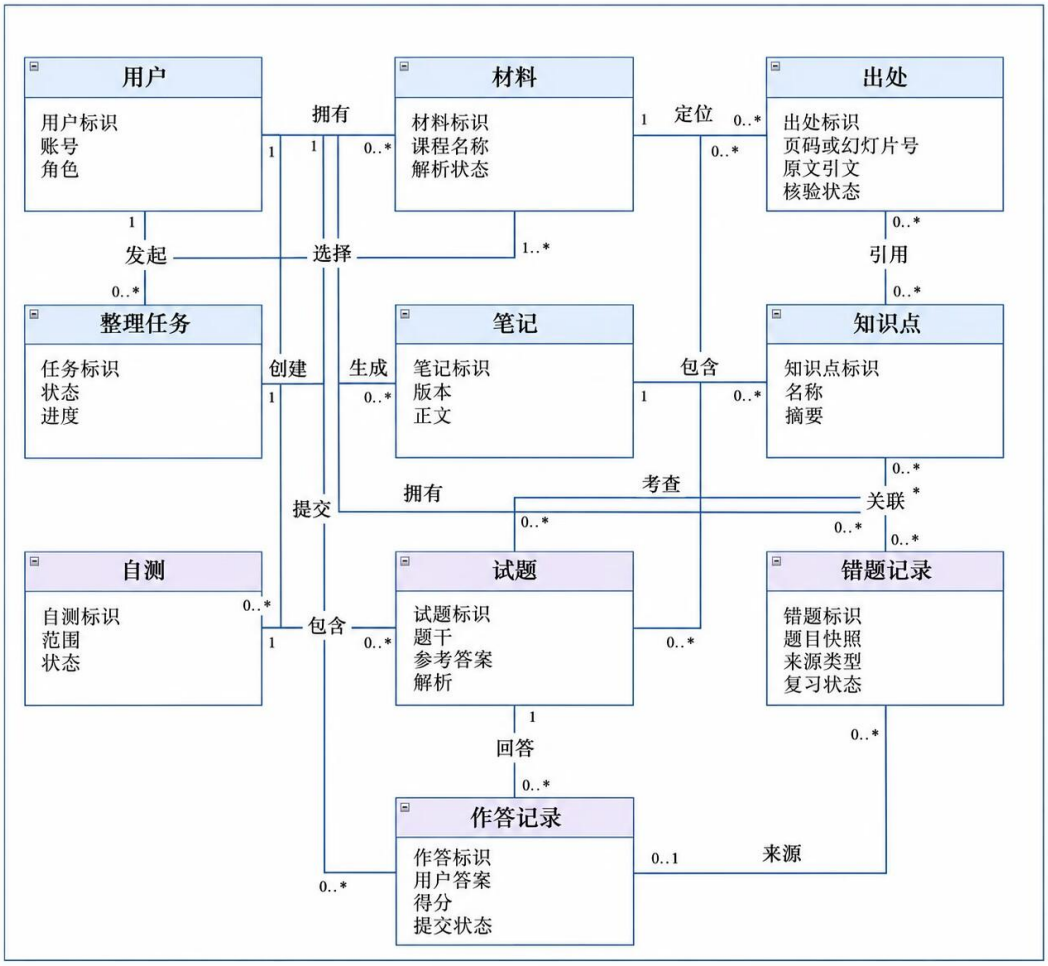


图 4 业务概念类图（实体与关系）

5 关键流程设计

5.1 上传与解析

- 前端先按 /api/meta 的格式白名单与大小上限做前置校验，减少无效请求。
- 后端再次校验（前端校验不可信），按文件头嗅探真实格式。
- 解析保留页码/幻灯片号与章节归属；平均每页少于 40 字符判为疑似扫描件。
- 扫描件在未配置 OCR 时返回明确的 warning，不声称解析成功。
- 解析结果入库并立即建 FTS5 索引，保证后续检索可用。

5.2 笔记生成

任务创建后立即返回 session_id（HTTP 202），实际生成在后台执行，前端轮询状态。这样长任务不会占住 HTTP 连接，也便于展示进度。

5.2.1 任务状态机

当前状态	触发	下一状态	说明
pending	后台开始执行	running	已入库，等待 worker 领取
running	管线正常结束	success	有结果、无警告
running	管线结束但有告警	partial	结果可用，warnings 非空
running	异常或无可解析内容	failed	error_message 记录原因
running	进程重启	failed	启动时把残留 running 置为失败，避免永久挂起

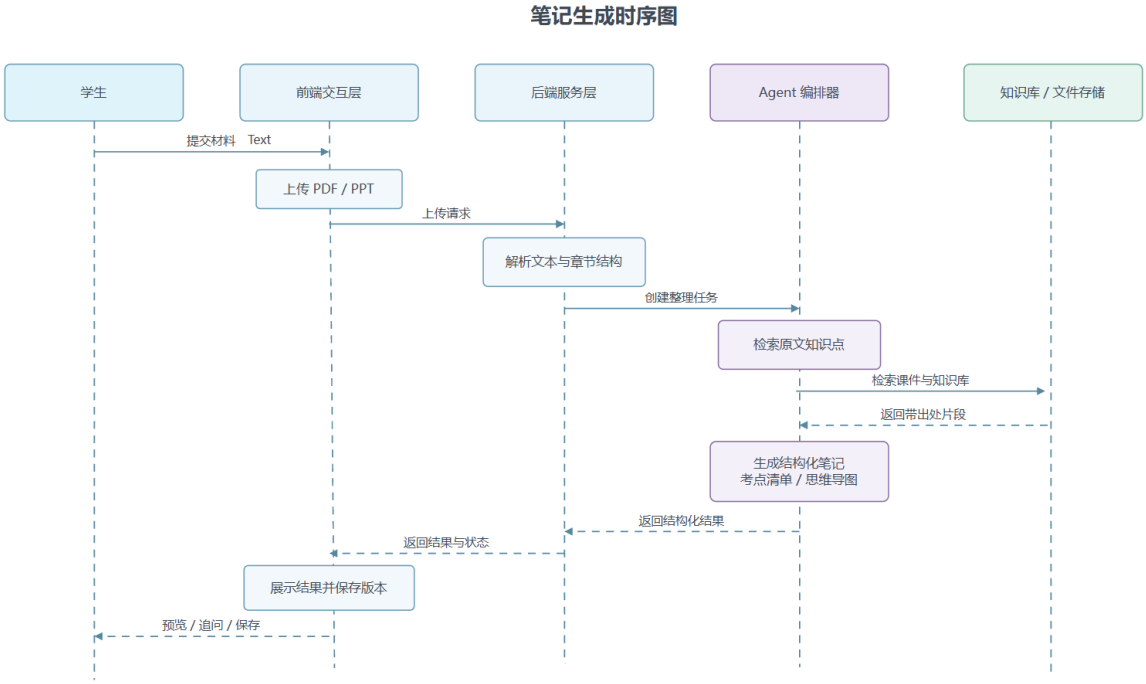

笔记生成时序图
学生
前端交互层
后端服务层
Agent 编排器
知识库 / 文件存储
提交材料 Text
上传 PDF / PPT
上传请求
解析文本与章节结构
创建整理任务
检索原文知识点
检索课件与知识库
返回带出处片段
生成结构化笔记
考点清单 / 思维导图
返回结构化结果
返回结果与状态
展示结果并保存版本
预览 / 追问 / 保存

图 5　笔记生成时序图

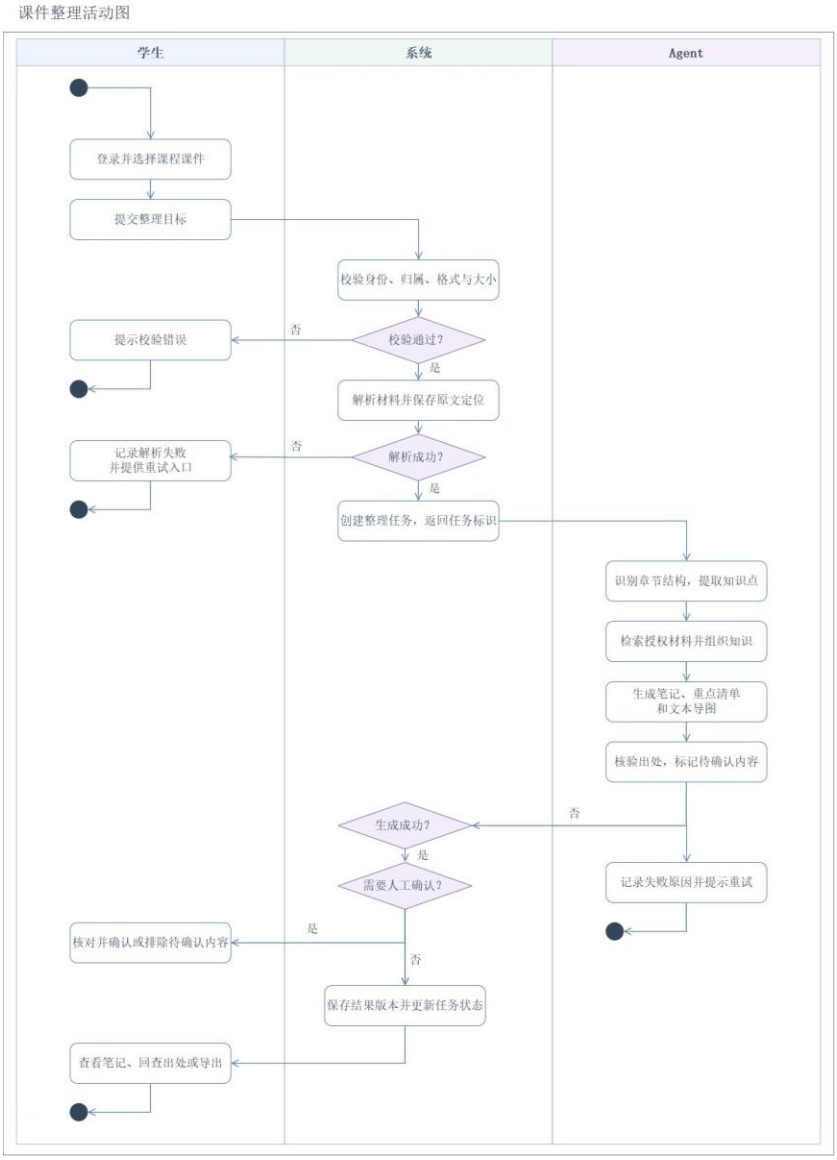


图 6 课件整理活动图（含校验与失败分支）

5.3 补充材料与合并

针对「后续还有材料进来」的场景：从历史任务或结果页进入追加模式，页面会列出该任务已有的材料并默认全选，用户可取消勾选写错或过时的材料。提交后以「勾选的旧材料 + 新材料」创建**新任务**，原任务保留不动。

行为	结果
新增材料的知识点	✓ 会被纳入（实测 1 条 → 2 条）
旧材料中的错误表述	！不会自动纠正；需取消勾选该材料
同名知识点	按名称归并，保留更长的定义，要点取并集
历史任务	✓ 原任务与新任务并存，可对比

设计取舍：系统不具备事实核验能力，因此不假装「自动纠错」，而是把材料范围的决定权交给用户，并保证每条结论都能点开原文核对。

5.4 课程问答

问答严格基于检索到的证据片段，并按三态决定行为：

检索状态	行为	返回标记
confident 非空	把片段交给模型作答，要求给出引用编号	insufficient=false
仅有 low_confidence	明确告知依据不足，建议换问法，不让模型自由发挥	insufficient=true
empty	直接回答「该内容在所选材料中未涉及」	insufficient=true
模型未给出可核验引用	拦截本次回答	insufficient=true

5.5 导出

结果页提供三种导出。导出内容与页面展示同源（同一次 render 结果），不是前端二次拼装，避免「页面看到的」和「导出的」不一致。

6 接口设计规范

6.1 响应包络

所有接口返回统一结构，前端封装层据此判断成功与否：

```
{
  "code": 0,            // 0 成功；非 0 为业务错误
  "message": "ok",
  "data": { }           // 成功时为业务数据，失败时为 null 或补充信息
}
```

6.2 错误约定

HTTP	code	场景
400	400	参数校验失败（格式、大小、必填项）
401	401	未登录或会话过期
404	404	资源不存在或不属于当前用户（越权也用 404）
409	409	状态冲突（任务未完成却索取结果）
422	422	请求体结构不合法
500	500	服务器内部错误（响应中不含堆栈，堆栈只进日志）

6.3 鉴权

登录成功签发签名会话，同时支持 Cookie 与 Authorization: Bearer 两种携带方式。Cookie 设 HttpOnly，避免 XSS 直接读取；Cookie 与 Bearer 双通道是为了兼容「浏览器直接访问」与「脚本调用」两种使用方式。

6.4 示例：创建任务

```
POST /api/sessions
Content-Type: application/json

{ "doc_ids": [1, 2], "course_name": "软件工程", "task_type": "note" }

→ 202
{ "code": 0, "message": "ok",
  "data": { "session_id": 11, "status": "pending", "progress": 0 } }
```

6.5 主要流程的调用顺序

以下为「上传 → 整理 → 取结果 → 导出」的最小调用序列，可作为接口测试与联调脚本的基准：

```
# 1) 登录，拿到 token（Cookie 亦可）
POST /api/auth/login          { "username": "...", "password": "..." }

# 2) 上传课件，得到 doc_id 与解析统计
POST /api/uploads             multipart/form-data: file, course_name
→ data.document.doc_id, parse_status, page_count, chunk_count

# 3) 创建整理任务（可一次带多份材料）
POST /api/sessions            { "doc_ids": [doc_id], "course_name": "..." }
→ data.session_id

# 4) 轮询任务状态（前端 1.5 秒一次）
GET  /api/sessions/{id}       → status, progress, stage_message

# 5) 取结果（含大纲、知识点、出处、质量自评）
GET  /api/sessions/{id}/result

# 6) 导出三种格式之一
GET  /api/sessions/{id}/export?fmt=docx|md|tex
```

注意：第 4 步若在未完成时直接调用第 5 步，会返回 409 而不是空结果——这一约定让前端可以明确区分「还在处理」与「真的没有内容」。

7 数据设计

7.1 设计原则

- 业务数据与领域知识分离：前者随用户增长，后者由管理员维护，分库避免互相影响。
- 所有用户资源带 user_id，查询一律带归属条件，越权在 SQL 层就被过滤。
- 时间统一存 UTC 字符串，展示时在前端本地化，避免跨时区错乱。
- 状态机只允许单向流转：pending → running → success/partial/failed，禁止回退。

7.2 关键约束

约束	说明
chunks 与 documents 级联删除	删除文档时同步清理分块与索引，避免悬挂证据
note_versions 唯一约束	同一笔记的版本号唯一（uq_note_versions）

约束	说明
FTS5 外挂表	ft_chunks 与 chunks 通过 rowid 关联，索引可重建
WAL 模式	读不阻塞写；备份需同时拷贝 -wal/-shm 或先停服务

7.3 备份与迁移

备份 = 停服务 + 打包 storage/ 与 .env；迁移到新服务器只需搬这三样（代码、.env、storage/），虚拟环境在新机重建。项目提供 migrate_backup.sh 自动完成停服、打包、排除虚拟环境与临时文件、恢复服务等步骤。

8 非功能设计

8.1 性能（线上实测）

场景	实测值	说明
6 页 / 4 MB 课件整理	12 秒	真实调用模型，共 6 次块级抽取
68 页全章整理	约 112 秒	71 次调用，49,167 tokens
单次全章成本	约 1~2 元	按 token 计费估算
非模型类接口	毫秒级	列表、历史、导出
首字节（公网）	约 58 毫秒	阿里云杭州节点

8.2 可靠性

- 后台任务 + 状态机：长任务不阻塞 HTTP，进程重启后未完成任务会被标记失败而非永久挂起。
- 块级容错：单块抽取失败只记告警，不使整单失败。
- systemd Restart=always：进程异常退出 3 秒后自动拉起。
- 导出兜底：渲染失败时用确定性模板输出，保证始终有结果可看。

8.3 安全

风险	措施
口令泄露	PBKDF2 加盐哈希存储，不存明文
越权访问	所有资源查询带 user_id；越权返回 404 不泄露存在性
密钥泄露	密钥只存服务端 .env；打包脚本扫描内容，发现真实密钥即拒绝出包
XSS	会话 Cookie 设 HttpOnly；前端不使用 v-html 渲染模型输出
上传攻击	格式按文件头嗅探、大小上限、解析异常统一捕获

8.4 可维护性

措施	效果
37 项自动化测试	覆盖契约、解析、接口、Agent 管线、配置回归
端到端自检脚本	一条命令验证 注册→上传→整理→结果→导出→检索→越权
SQL 参数静态检查	防止占位符与参数数量不一致（曾出过 3 处）
测试存储隔离	跑测试使用临时目录，不会清空真实数据库（曾误清两次）
线上验证脚本	直接打生产地址走完整流程，验证「老师打开后能用」

9 部署与运维

9.1 部署拓扑

```
用户浏览器
   │  HTTP :8000
   ▼
阿里云 ECS（Ubuntu 22.04）
   ├─ systemd: course-note-agent.service
   │     └─ uvicorn (单 worker) :8000
   │           ├─ /api/*        后端接口
   │           └─ /*            Vue 构建产物（同源托管）
   ├─ storage/business.db   业务库（WAL）
   ├─ storage/kb.db         知识库
   ├─ storage/uploads/      原始课件
   └─ .env                  密钥与开关（chmod 600）
```

9.2 运维要点

- 改配置后必须 systemctl restart 才生效（配置在进程启动时读取）。
- 日志：journalctl -u course-note-agent 或 logs/service.log。
- 云厂商安全组需放行 TCP 8000，否则「本机通、外网不通」。
- 迁移换机：migrate_backup.sh 打包 → 新机解压 → deploy.sh 一键部署。
- 单 worker 是刻意选择：SQLite 写入与内存占用可控，代价是并发能力有限。

10 关键设计决策

决策	选择	理由	代价
检索方案	FTS5 + 二元组覆盖率，不用向量库	中文无需分词；结果可解释；零额外服务	语义召回弱，同义改写易漏
证据策略	每条结论必须回查原文，查不到即丢弃	从机制上杜绝编造，而不是靠提示词约束	召回率下降，dropped_items 会非零
模型调用	按块抽取而非整篇投喂	单块失败可控；长课件不超上下文	调用次数多，成本上升
并发模型	单 worker + 后台任务	SQLite 单机场景够用；内存占用稳定	同时只能处理一个重任务
导出格式	Word + Markdown + LaTeX	分别对应交作业、贴博客、排版公式三种真实场景	三套渲染需各自测试
配置开关	功能以 ENABLE_* 控制，默认关闭未稳定项	避免半成品对外暴露	配置多了容易漏改（曾因注释导致开关全失效）

11 已知限制与演进方向

11.1 已知限制

限制	影响	现状
无事实核验能力	多份材料冲突时并列保留，不判断对错	！设计上如实呈现，由用户取舍
领域知识库为空	问答只基于上传课件	！表结构已建，未灌数据
智能自测未实现	用例图中的自测/错题本不可用	！仅设计稿
向量检索关闭	同义改写问题召回不足	！配置项保留
单 worker	并发整理会排队	！与 SQLite 匹配的取舍
扫描件依赖 OCR	未配密钥时无法识别图片型课件	！引入时应加超时与失败兜底

11.2 演进方向

- 填充领域知识库并与课件检索融合，支持跨课程知识关联。
- 引入向量检索作为补充召回，与二元组结果做融合排序。
- 实现智能自测与错题本（表结构已预留）。
- 任务队列化（如 Redis + worker），解除单 worker 并发限制。
- 增加管理端可视化：解析质量、丢弃率、成本统计。

附：智能自测与错题收录的功能设计（本期未实现，仅保留设计稿）

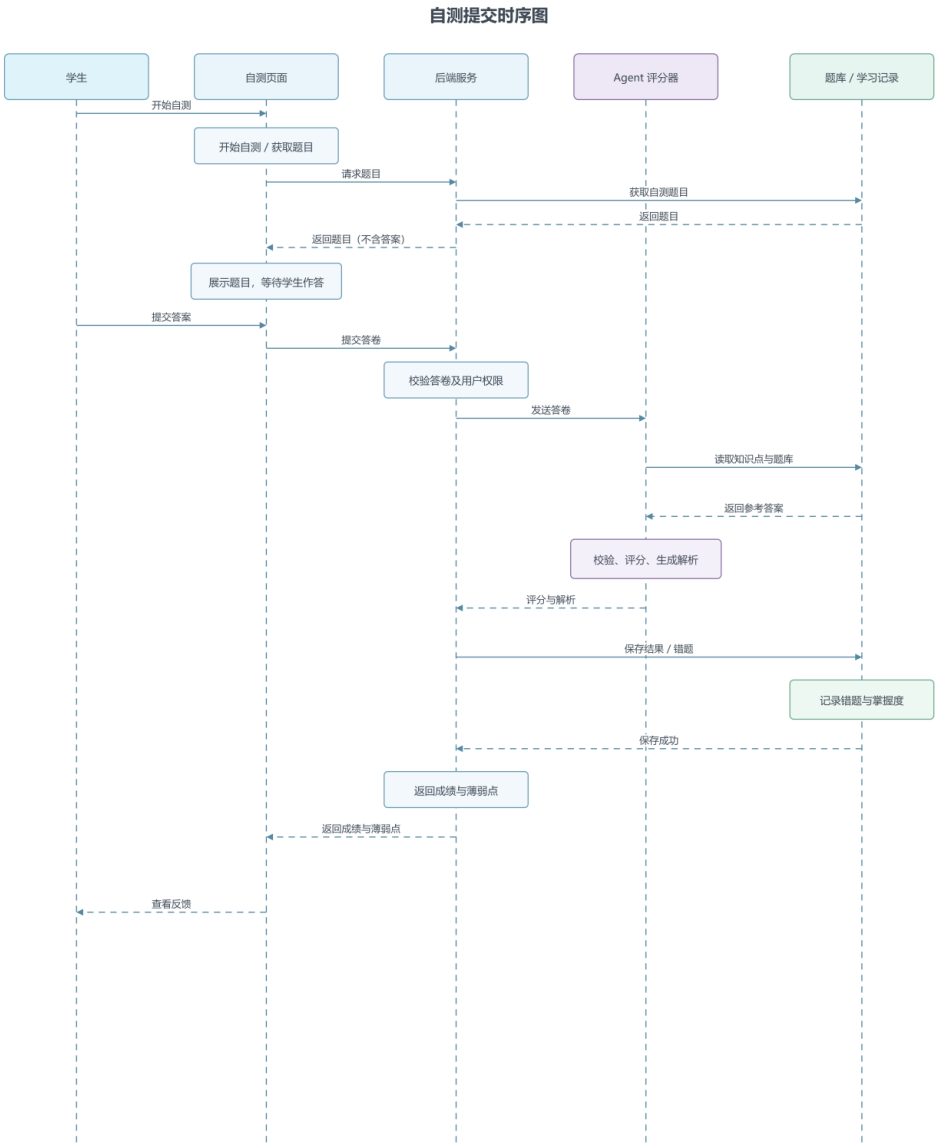


图 7　自测提交时序图（设计稿，未实现）

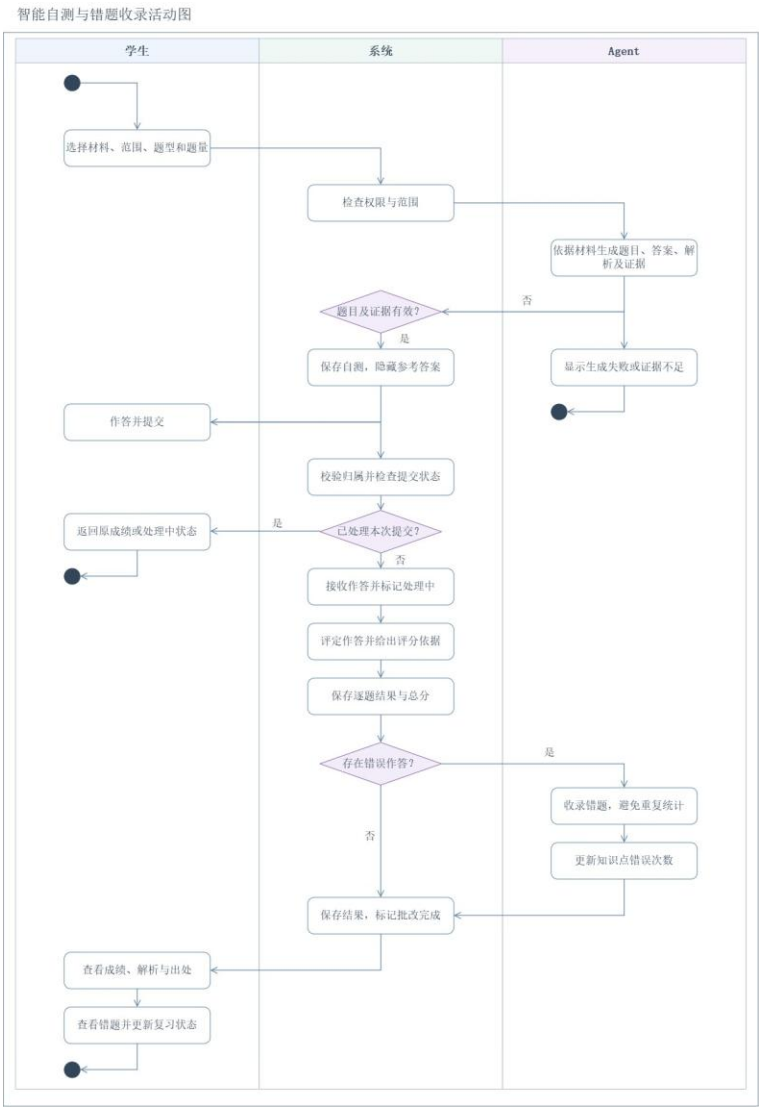


图 8　智能自测与错题收录活动图（设计稿，未实现）

12 附录

12.1 配置项（共 24 项，节选）

配置项	默认值	说明
ALLOWED_EXTENSIONS	16 种	允许上传的扩展名，逗号分隔
MAX_UPLOAD_MB	50	单文件大小上限
LLM_API_KEYS	（空）	支持多把密钥轮换
LLM_MODEL	deepseek-chat	模型名
LLM_TIMEOUT / MAX_RETRIES	120 / 3	单次调用超时与重试
ENABLE_QUALITY_EVAL	1	质量自评开关
ENABLE_VECTOR	0	向量检索开关（当前关闭）
SECRET_KEY	（部署时随机生成）	会话签名密钥，泄露等于可伪造登录

注意：.env 中一旦写了行内注释（KEY=1 # 说明），值会连注释一起被读入，布尔开关将静默失效。代码已做剥离处理，但新增配置项时要留意。

12.2 质量自评口径

维度	权重	含义
核心功能完成度 core	85%	有原文出处的知识点占比、抽取完整性
结构化与规范性 structure	10%	是否有大纲、重点清单、思维导图
拓展关联 extension	5%	是否建立跨章节关联

12.3 效果数据（可引用）

指标	本系统	裸模型
知识点覆盖率	66.67%（24/36）	38.89%（14/36）
无依据陈述比例	0%	100%
问答正确率	83.33%（5/6）	未测
质量自评	82.1 / 100	—

口径说明：覆盖率为保守文本匹配结果，未匹配项中含近义表述，不得表述为人工语义覆盖率。

—— 文档结束 ——